# RAMDEOBABA UNIVERSITY, NAGPUR

## DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

### B.Tech. CSE (Artificial Intelligence and Data Science) (AIDS)

**SEMESTER V**

**SI LAB**

**PROJECT REPORT**

# NEW CAR PRICE PREDICTION SYSTEM

**Submitted by**

| Student Name | Roll No. | Enrollment No. |
|---|---|---|
| Ojas Thakre | 51 | 24RBU/UG/CS/02/000088 |

**Academic Year: 2026–27**

NEW CAR PRICE PREDICTION SYSTEM

# SI Lab Project Report

## 1. PROJECT OBJECTIVE

The main purpose of this project is to apply statistical inference and machine learning to the prices of new cars sold in India, and to build a regression model that estimates the ex-showroom price (in Indian rupees, INR) of a car variant from its specifications such as engine displacement, power, torque, dimensions, mileage, fuel type, body type, transmission type and brand. The problem addressed is that a car's price depends on many interacting specifications, so a data-driven model trained on recorded prices can give a quick and consistent estimate, while statistical tests show which specifications are genuinely related to price. The expected output is a predicted ex-showroom price, and the saved model is intended to be used in a Streamlit application as a decision-support tool for students, researchers and buyers comparing car variants.

Since the ex-showroom price is a continuous numeric quantity, this project is a regression problem. The classification metrics suggested in the report format (accuracy, precision, recall, F1-score, confusion matrix) are therefore replaced by the regression metrics MAE, MSE, RMSE and $R^2$. The objectives are:

- Load and explore the new-car dataset (*cars_ds_final.csv*).
- Clean the data (price text to numbers, unit-bearing specifications to numeric values, duplicates, missing values) and prepare it for model building.
- Apply descriptive statistics, probability distributions, sampling, the Central Limit Theorem, confidence intervals, hypothesis tests and correlation analysis to the price data.
- Train multiple regression models: Linear, Ridge, Lasso, Decision Tree and Random Forest.
- Evaluate and compare model performance using MAE, MSE, RMSE and $R^2$, and test actual versus predicted prices with a paired t-test.
- Automatically select the best model and save it, with its supporting files, to disk.
- Prepare the exported files for a Streamlit GUI that accepts car specifications and displays the predicted price.

## 2. INTRODUCTION

The price of a new car is influenced by its engine, size, features, fuel type, transmission and brand positioning. Buyers, dealers and analysts often need a reasonable estimate of what a variant with a given set of specifications should cost. Traditionally this comparison is done by hand from brochures and price lists, which is slow and inconsistent. With a dataset of recorded variants and their ex-showroom prices, statistical methods can quantify how strongly each specification is related to price, and machine-learning models can learn these relationships and produce quick estimates.

The motivation behind this project is to apply the statistical-inference and machine-learning techniques studied in the SI Lab to a practical, familiar problem, and to prepare the result in a form that can be used by an easy-to-use application rather than only as a notebook.

## 2.1 Problem Statement

To develop a regression-based system that predicts the new-car ex-showroom price (INR) for a given set of car specifications, by analysing the price data with statistical inference, training and comparing several regression models, and saving the best one for deployment through a graphical interface.

## 2.2 Scope of the Project

The project includes data loading and cleaning, preprocessing through a reusable pipeline, descriptive and inferential statistical analysis, training and comparison of five regression models, automatic selection and saving of the best model, and the export of configuration files for a Streamlit web interface. The limitations are: the dataset contains 1,276 new-car variants and predicts the *historical ex-showroom prices recorded in the dataset*; it does not predict used-car resale prices or current live showroom quotations; many specification columns are sparse (for example, Electric_Range is available for only 17 variants); some recorded values are inconsistent in units; and the model can only be reliable for brands, models and specifications similar to those in the data. Predictions are estimates, not quotations.

# 3. PROJECT METHODOLOGY

The project follows the stages of dataset loading and exploration (including statistical inference), preprocessing, feature selection, training of multiple models, performance evaluation, and best-model selection, followed by preparation of the files for the GUI. The work was carried out in Google Colab; the frontend is developed with Streamlit in VS Code.

## 3.1 Dataset Loading and Exploration

- **Dataset:** Indian new-car dataset (file cars_ds_final.csv) containing car models and their variants with technical specifications and ex-showroom prices (see References).
- **Size:** 1,276 rows and 141 columns (including an index column, “Unnamed: 0”, that carries no information). The data covers 263 car models and 39 distinct makes; the Make field is missing for 75 records.
- **Target variable:** Ex-Showroom_Price, stored as text such as “Rs. 2,92,667” and converted to a numeric INR value.
- **Features:** 30 selected specifications: 21 numeric (for example Displacement, Cylinders, Power, Torque, Kerb_Weight, Wheelbase, mileage and Number_of_Airbags) and 9 categorical (Make, Model, Variant, Fuel_Type, Body_Type, Type, Drivetrain, Cylinder_Configuration, Emission_Norm).

Table 1 lists the selected columns with their data types after cleaning and Table 2 gives summary statistics computed from the dataset.

| Column(s) | Data type | Description |
|---|---|---|
| Ex-Showroom_Price | object → float64 | Target: ex-showroom price in INR |
| Make, Model, Variant | object (string) | Brand, car model and variant name |
| Fuel_Type, Body_Type, Type | object (string) | Fuel, body style, transmission type |
| Drivetrain, Cylinder_Configuration, Emission_Norm | object (string) | Drive layout, cylinder layout, emission standard |
| Displacement, Cylinders, Valves_Per_Cylinder | float64 | Engine size (cc) and cylinder details |
| Power, Torque | float64 | Maximum power and torque (numeric part) |
| Length, Width, Height, Wheelbase, Ground_Clearance | float64 | Dimensions (mm) |
| Kerb_Weight, Fuel_Tank_Capacity, Boot_Space | float64 | Weight and capacities |
| City_Mileage, Highway_Mileage, ARAI_Certified_Mileage | float64 | Fuel-efficiency figures |
| Doors, Seating_Capacity, Gears, Number_of_Airbags | float64 | Layout and safety counts |
| Electric_Range | float64 | Range of electric variants (km) |

*Table 1: Selected dataset columns and data types*

| Statistic | Displacement | Power | Torque | Seating | Price (INR) |
|---|---|---|---|---|---|
| Count | 1,264 | 1,276 | 1,274 | 1,270 | 1,276 |
| Mean | 1,858.81 | 156.54 | 266.52 | 5.27 | 4,596,537.89 |
| Std. dev. | 1,063.15 | 136.60 | 185.48 | 1.15 | 12,147,346.94 |
| Minimum | 72 | 12 | 8.3 | 2 | 236,447 |
| Median | 1,497 | 110 | 209 | 5 | 1,060,064.50 |
| Maximum | 7,993 | 1,600 | 1,712 | 16 | 212,155,397 |

*Table 2: Summary statistics of key numeric columns*

**Missing values and duplicates:** the dataset has no duplicate rows and the price column has no missing values, but many specification columns do: for example Electric_Range is missing for 1,259 of 1,276 rows, Highway_Mileage for 800, City_Mileage for 555, Kerb_Weight for 365, Ground_Clearance for 289 and Boot_Space for 249. These are handled inside the model pipeline by imputation rather than by dropping rows. A few columns also contain implausible values (for example a maximum City_Mileage of 26,032 and a minimum Height of 1.845 against typical values near 1,500 mm), which indicate unit or entry inconsistencies in the source data (see Section 6.2).

**Exploratory visualizations.** Figure 1 shows that the price distribution is extremely right-skewed: most variants lie below about Rs. 30 lakh, while a few supercars reach over Rs. 20 crore, and the boxplot shows many high outliers. Figure 2 shows that price rises with displacement, power, torque and kerb weight, with the two highest-priced cars standing far above the rest. Figure 3 shows that median price differs greatly between makes, fuel types, transmission types and body types (for example hybrid and electric cars, automatic and DCT transmissions, and sports and coupé bodies have the highest medians). Figure 4 shows the correlation heatmap of the numeric columns.

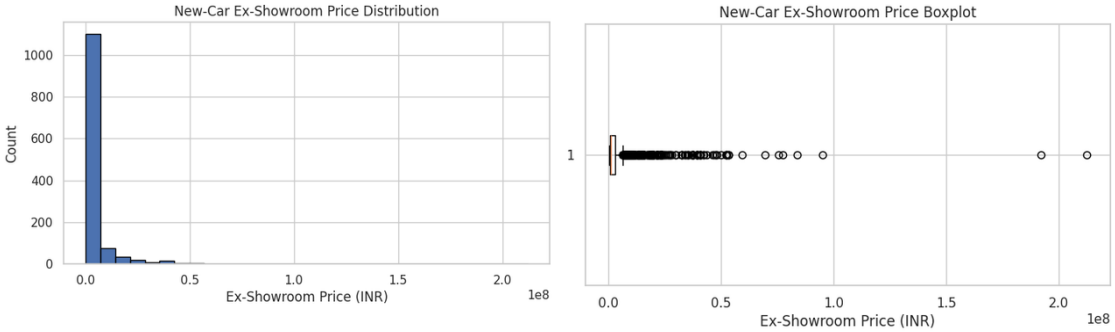



*Figure 1: Distribution and boxplot of new-car ex-showroom prices (INR)*

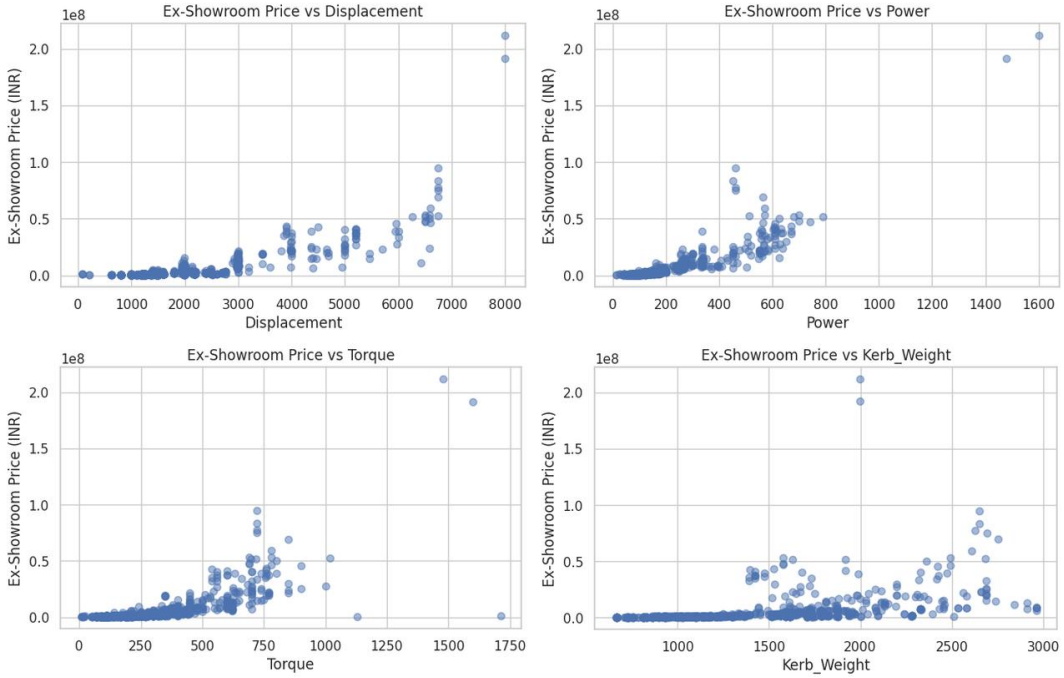



*Figure 2: Ex-showroom price versus Displacement, Power, Torque and Kerb_Weight*

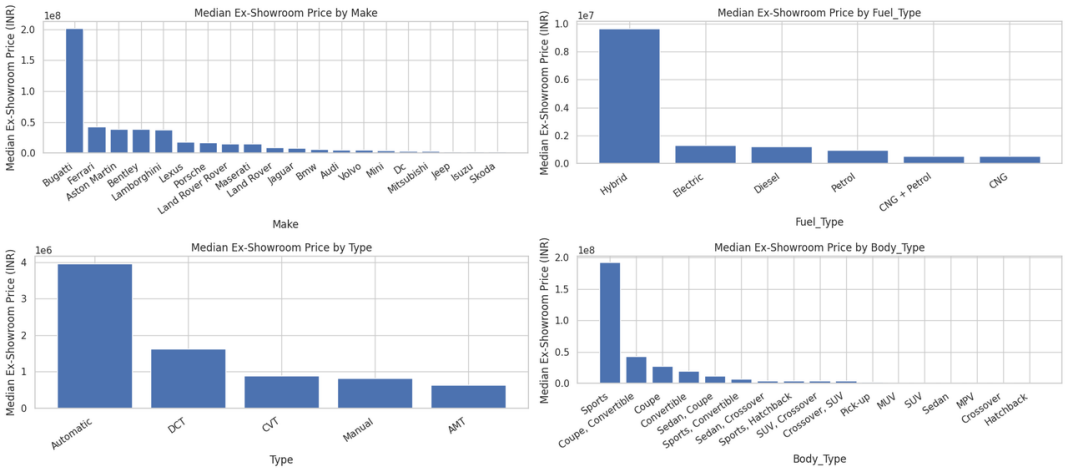



*Figure 3: Median ex-showroom price by Make, Fuel_Type, Type and Body_Type*

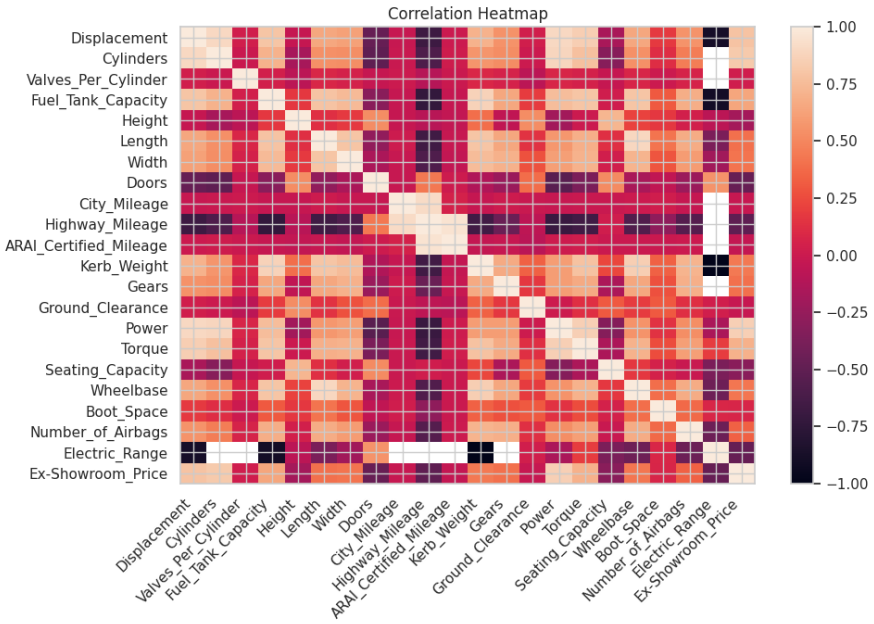



*Figure 4: Correlation heatmap of numeric specifications and price*

**Sampling, Central Limit Theorem and standard error.** To demonstrate probability distributions, uniform and binomial samples were simulated (these are demonstration values, not car data). For the Central Limit Theorem, 1,000 random samples of 30 prices (with replacement) were drawn and their means plotted (Figure 5); because of the extreme luxury prices, the sampling distribution is still right-skewed at n = 30 although it is much more compact than the raw prices. A simple random sample of 100 records and a fuel-type stratified sample (up to 20 records per fuel type, 91 records in total) were also drawn. The standard error of the mean price is Rs. 340,060.43.

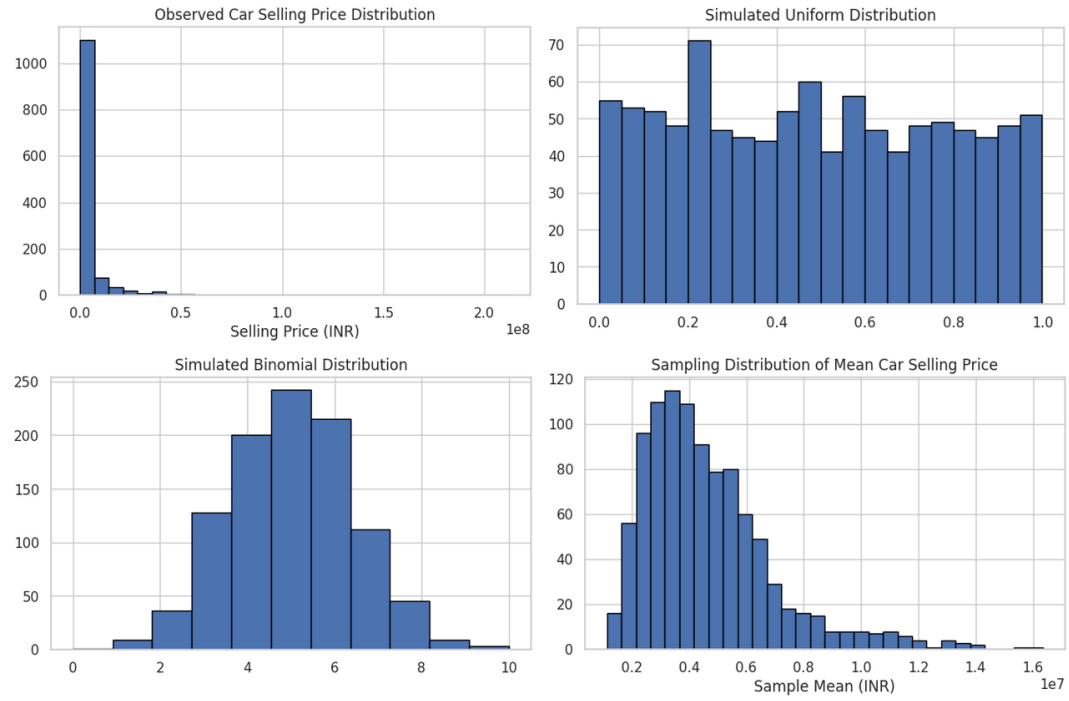



*Figure 5: Observed price distribution, simulated uniform and binomial distributions, and sampling distribution of the mean price*

**Confidence interval.** Using the t-distribution, the 95 % confidence interval for the mean ex-showroom price is Rs. 3,929,398.38 to Rs. 5,263,677.39 around a sample mean of Rs. 4,596,537.89. This interval estimates the population mean; it does not mean that 95 % of individual car prices lie inside it.

**Hypothesis tests (α = 0.05).** Four tests were performed (Table 3). In every case the p-value was below 0.00001, so the null hypothesis was rejected.

| Test | Null hypothesis (H0) | Statistic | Decision |
|---|---|---|---|
| One-sample t-test | Mean price = Rs. 10,00,000 | t = 10.576 | Reject H0 |
| Independent two-sample t-test (Welch) | Mean price is equal for manual and automatic cars | t = −11.425 | Reject H0 |
| One-way ANOVA | Mean price is equal across fuel types | F = 7.179 | Reject H0 |
| Chi-square test of independence | Fuel type and transmission type are independent | $\chi^2 = 76.832$ | Reject H0 |

*Table 3: Hypothesis tests on new-car ex-showroom prices ($p < 0.00001$ for all)*

## 3.2 Data Preprocessing

- Exact duplicate rows were checked (none were found) and the exported index column “Unnamed: 0” was removed.
- The target price text such as “Rs. 2,92,667” was converted to a number by removing “Rs.” and the commas; rows with a missing or non-positive price were dropped. All 1,276 records remained.
- Numeric specifications that contain units were converted by extracting the first number with a regular expression (for example “624 cc” → 624, “23.6 km/litre” → 23.6, “38PS@5500rpm” → 38).
- Numeric features (21 columns) were passed through a median SimpleImputer and then standardised with StandardScaler.
- Categorical features (9 columns) were passed through a most-frequent SimpleImputer and then converted with OneHotEncoder(handle_unknown="ignore").
- Input features X (30 columns) were separated from the target y (Ex-Showroom_Price).
- The data was split into 80 % training (1,020 rows) and 20 % testing (256 rows) with random_state = 42.

All these steps are wrapped in a scikit-learn ColumnTransformer inside a Pipeline, so the same preprocessing is automatically applied to new inputs at prediction time and no separate scaler or encoder file is needed.

## 3.3 Feature Extraction / Selection

No separate feature-extraction technique was required because the dataset already provides meaningful specification attributes. Thirty attributes (21 numeric and 9 categorical) were selected as input features; the index column was not used. Categorical attributes are represented by one-hot encoding and numeric attributes by standardised values. Correlation analysis supports this selection (Table 4). Power (Pearson r = 0.855), Cylinders (0.821), Displacement (0.794) and Torque (0.702) have the strongest linear correlation with price, while the Spearman rank correlations are higher for most size and performance features (Power 0.930, Torque 0.896, Displacement 0.877, Kerb_Weight

0.866), showing a strong but non-linear monotonic relationship. Valves_Per_Cylinder, City_Mileage, ARAI_Certified_Mileage and Ground_Clearance have no significant linear correlation with price. Several features are strongly correlated with each other (for example Displacement and Cylinders, r = 0.899; Displacement and Power, r = 0.901). Correlation does not prove causation.

| Feature vs price | Pearson r | Pearson p-value | Spearman ρ |
|---|---|---|---|
| Power | 0.855 | < 0.00001 | 0.930 |
| Cylinders | 0.821 | < 0.00001 | 0.731 |
| Displacement | 0.794 | < 0.00001 | 0.877 |
| Torque | 0.702 | < 0.00001 | 0.896 |
| Fuel_Tank_Capacity | 0.643 | < 0.00001 | 0.853 |
| Kerb_Weight | 0.438 | < 0.00001 | 0.866 |
| Length | 0.401 | < 0.00001 | 0.865 |
| Highway_Mileage | −0.508 | < 0.00001 | −0.609 |
| Doors | −0.462 | < 0.00001 | −0.298 |
| Valves_Per_Cylinder | 0.030 | 0.29812 | 0.123 |
| City_Mileage | −0.016 | 0.66636 | −0.605 |
| ARAI_Certified_Mileage | −0.036 | 0.22450 | −0.615 |

*Table 4: Selected Pearson and Spearman correlations with ex-showroom price*

## 3.4 Training Multiple Regression Models

Because the target is numeric, five regression models were trained on the same training set using identical preprocessing.

| Sr. No. | Model Name | Purpose / Notes |
|---|---|---|
| 1 | Linear Regression | Baseline model that fits a linear relationship between the specifications and price. |
| 2 | Ridge Regression (alpha = 1.0) | Linear model with L2 regularisation to reduce overfitting and handle correlated features. |
| 3 | Lasso Regression (alpha = 0.1) | Linear model with L1 regularisation that can shrink weak coefficients towards zero. |
| 4 | Decision Tree (max_depth = 8) | Non-linear model that splits the data on feature thresholds; captures interactions. |
| 5 | Random Forest (200 trees, max_depth = 12) | Ensemble of decision trees that averages their predictions; robust on non-linear data. |

*Table 5: Regression models implemented*

## 3.5 Performance Evaluation and Model Comparison

Since this is a regression task, the models were evaluated on the 20 % test set using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and the $R^2$ score. Lower MAE, MSE and RMSE and higher $R^2$ indicate a better model. A confusion matrix, accuracy, precision, recall and F1-score apply only to classification and are therefore not used; instead, an actual-versus-predicted plot is provided (Figure 16).

| Model | MAE (INR) | MSE | RMSE (INR) | $R^2$ Score |
|---|---|---|---|---|
| Decision Tree | 935,334.29 | $9.93 \times 10^{12}$ | 3,151,180.20 | 0.9485 |
| Random Forest | 783,042.22 | $1.12 \times 10^{13}$ | 3,350,612.14 | 0.94 |
| Ridge Regression | 1,707,850.34 | $1.29 \times 10^{13}$ | 3,595,280.95 | 0.93 |
| Linear Regression | 1,830,025.15 | $2.29 \times 10^{13}$ | 4,788,228.16 | 0.88 |
| Lasso Regression | 1,991,447.13 | $3.17 \times 10^{13}$ | 5,629,418.13 | 0.84 |

*Table 6: Model comparison on the test set (sorted by $R^2$ score)*

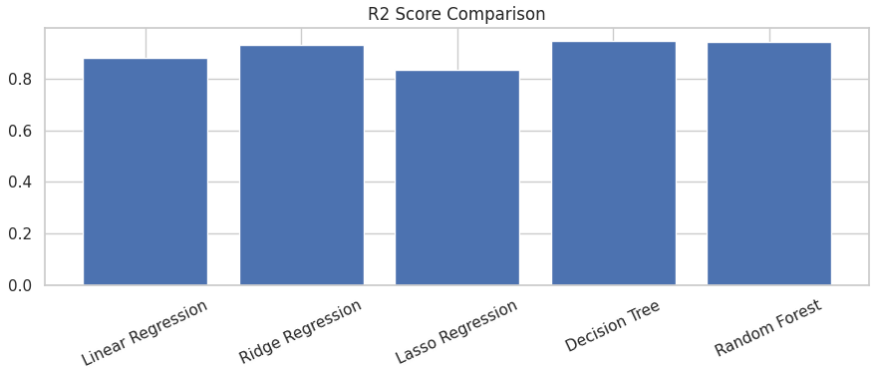



*Figure 6: $R^2$ score comparison across models*

**Paired t-test.** Each actual test price was paired with the prediction of the best model for the same variant. The mean actual price was Rs. 4,166,582.38 and the mean predicted price was Rs. 4,394,532.69 ($t = -1.158$, $p = 0.24787$), so there is insufficient evidence of a mean difference at $\alpha = 0.05$. This test does not replace MAE and RMSE, and a non-significant result does not by itself prove that the model is accurate.

### 3.6 Best Model Selection

The notebook selects the model with the highest $R^2$ score on the test set. The Decision Tree (max_depth = 8) was selected, with $R^2 = 0.9485$, MAE = 935,334.29 INR and RMSE = 3,151,180.20 INR. It also has the lowest RMSE, which is about 6 % lower than the Random Forest (3,350,612.14), about 12 % lower than Ridge Regression and about 34 % lower than Linear Regression (4,788,228.16). The Random Forest is very close ($R^2 = 0.94$) and actually has the lowest MAE (783,042.22 INR), so the two tree-based models are nearly equivalent on this test set. The three linear models perform clearly worse, which shows that the relationship between specifications and price is non-linear. The selected pipeline (preprocessing + model) is saved as new_car_price_model.pkl using joblib, together with best_car_model_name.txt, car_model_features.json, car_feature_config.json, sklearn_version.txt, cars_cleaned.csv and car_model_comparison.csv.

# 4. PROJECT IMPLEMENTATION

## 4.1 Tools and Technologies

| Tool / Technology | Version | Purpose |
|---|---|---|
| Python (Google Colab) | Colab runtime | Programming language and development environment |
| VS Code + Streamlit | — | Frontend development and GUI / web interface |
| Pandas / NumPy | — | Data handling |
| SciPy (stats) | — | Statistical tests, confidence intervals, correlation |
| Scikit-learn | 1.6.1 | Preprocessing, models and metrics |
| Matplotlib / Seaborn | — | Visualization |
| Joblib / JSON | — | Saving the trained model and input configuration |

*Table 7: Tools and technologies*

## 4.2 Module-wise Implementation

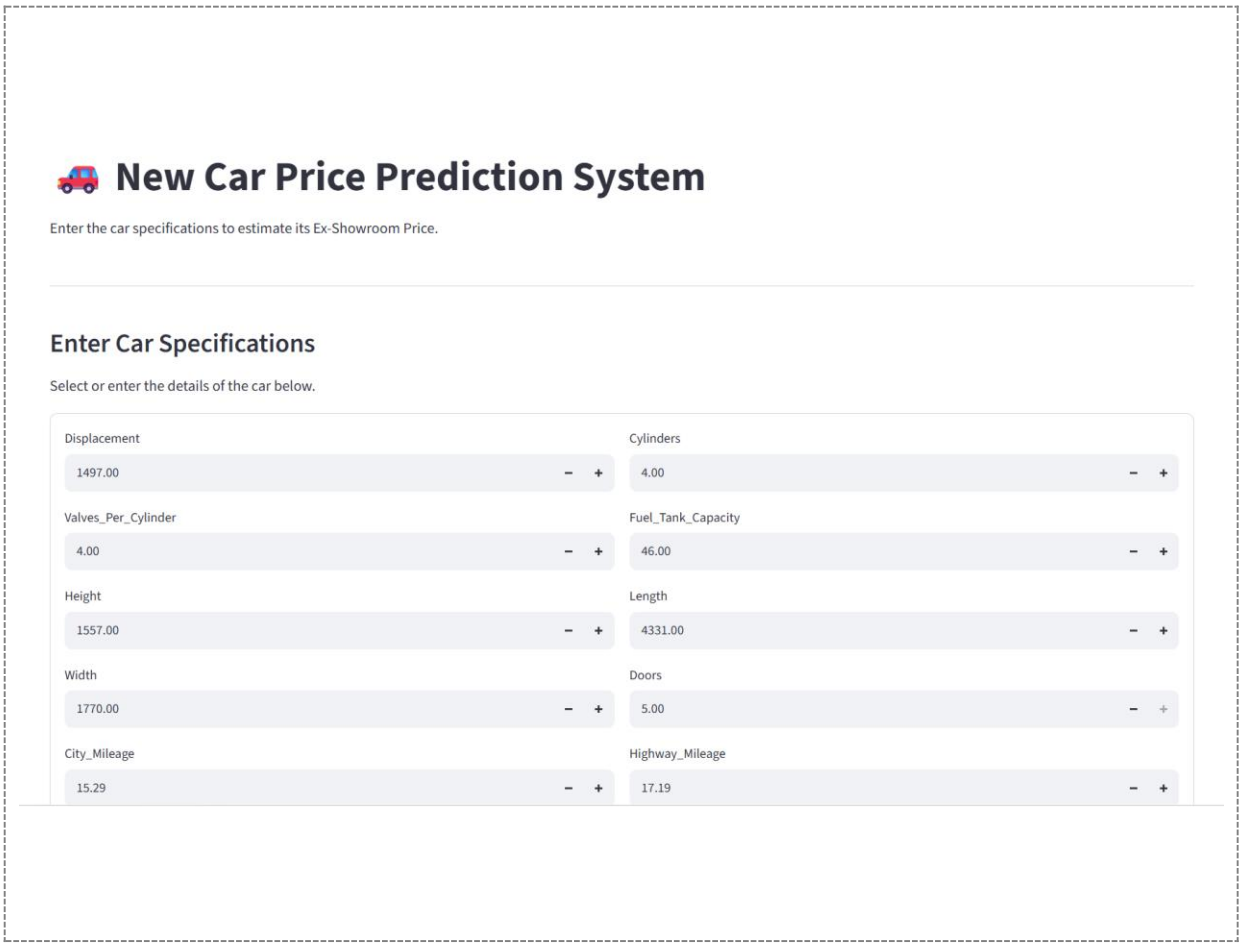
- **Dataset loading and exploration module:** reads cars_ds_final.csv (uploaded in Colab), prints the shape, column names, first rows, missing values, duplicates and a descriptive summary.
- **Data cleaning and preprocessing module:** removes duplicates and the index column, converts the price text and unit-bearing specifications to numbers, and defines a ColumnTransformer with a numeric pipeline (median imputation and standard scaling) and a categorical pipeline (most-frequent imputation and one-hot encoding).
- **Statistical inference module:** computes descriptive statistics, simulated distributions, random and stratified samples, standard error, the 95 % confidence interval, the four hypothesis tests and the Pearson/Spearman correlations.
- **Feature selection module:** the lists numeric_features and categorical_features define the 30 input features; no extra extraction was needed.
- **Model training and evaluation module:** performs the 80/20 split, trains the five pipelines, computes MAE, MSE, RMSE and $R^2$, and runs the paired t-test on actual versus predicted prices.
- **Best model saving module:** the model with the highest $R^2$ is saved with joblib.dump() to new_car_price_model.pkl; the feature order, input configuration, scikit-learn version, cleaned data and comparison table are saved as JSON, TXT and CSV files and downloaded from Colab.
- **GUI input and prediction module:** the Streamlit app loads the saved pipeline and configuration, accepts car specifications, builds a one-row DataFrame and displays the predicted price.

## 4.3 GUI Implementation

The model is trained in Google Colab and the interface is developed with Streamlit in VS Code. The exported files are placed in the same folder as app.py: new_car_price_model.pkl (the fitted preprocessing and prediction pipeline), car_feature_config.json (feature names, numeric ranges and category choices for the input controls), car_model_features.json (exact feature order used in training), sklearn_version.txt (the scikit-learn version, 1.6.1, that must be installed to load the model), cars_cleaned.csv, car_model_comparison.csv and best_car_model_name.txt.

The interface is driven by car_feature_config.json. For every numeric feature the configuration stores the minimum, maximum and default (median) value, which are used for the number inputs; for every

categorical feature (Make, Model, Variant, Fuel_Type, Body_Type, Type, Drivetrain, Cylinder_Configuration and Emission_Norm) it stores the list of available options for the drop-down lists. When the user clicks the predict button, the inputs are converted into a DataFrame with the same columns and order used in training and passed to the saved pipeline. The pipeline applies imputation, scaling and encoding and returns the predicted ex-showroom price in INR. The result is an estimate of the historical ex-showroom price in the dataset and not a live dealer quotation. The GUI is started with the command: streamlit run app.py.



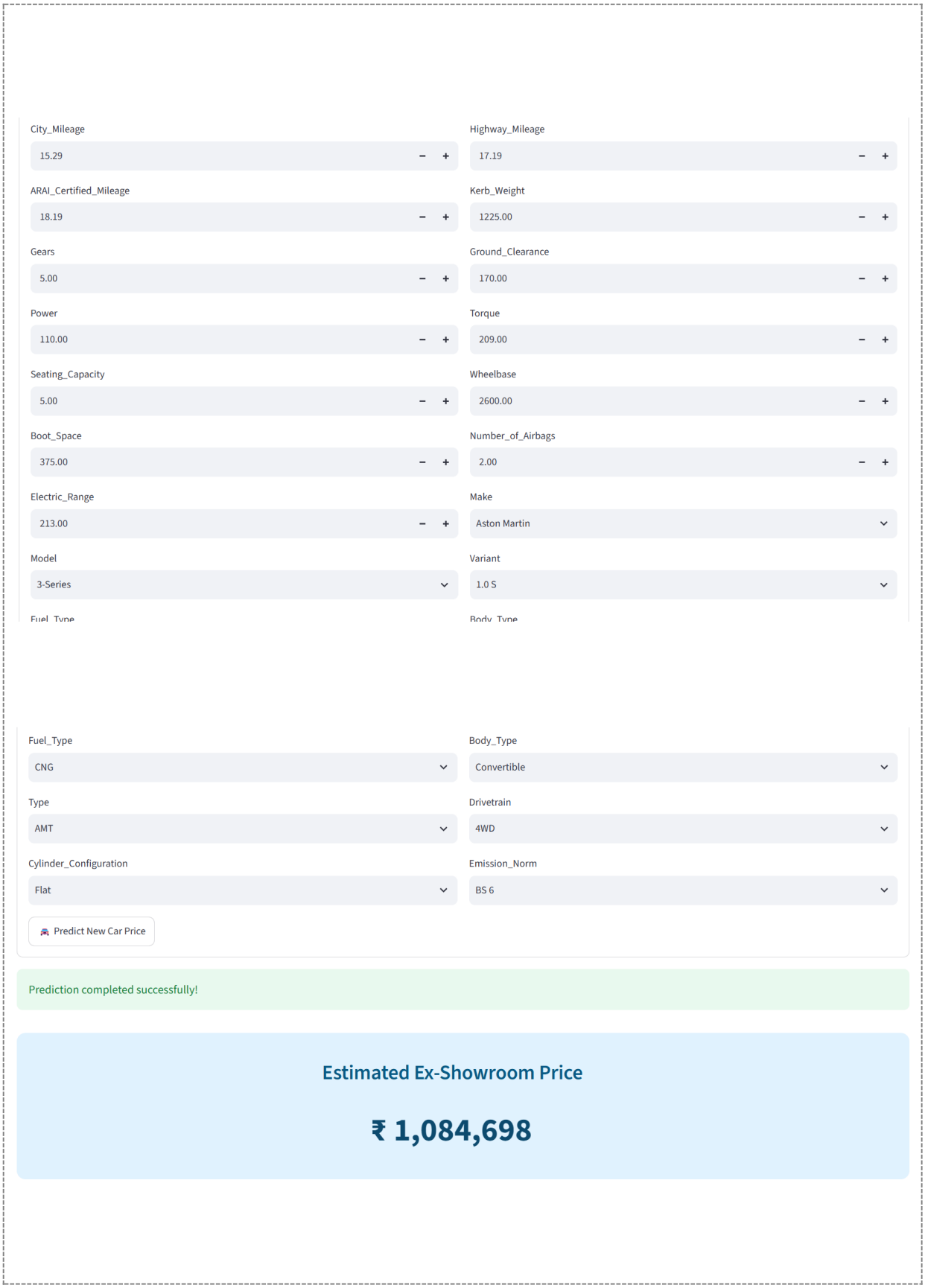



*Figure 7: GUI interface and prediction result (Streamlit)*

## 5. COMPLETE CODE

The complete source code of the project (Google Colab notebook, New_Car_Price_Prediction_Statistical_Inference.ipynb) is given below. It contains data loading, cleaning, statistical analysis, training of all models, best-model selection and saving, and the download of the exported files. It is run in Google Colab with cars_ds_final.csv uploaded when requested.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import json
import os
import re
import sklearn


from scipy import stats
from IPython.display import display


from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


sns.set_theme(style="whitegrid")


# Load the uploaded new-car dataset
DATA_FILE = "cars_ds_final.csv"


if not os.path.exists(DATA_FILE):
    from google.colab import files
    uploaded = files.upload()
    csv_files = [name for name in uploaded if name.lower().endswith(".csv")]
    if not csv_files:
        raise FileNotFoundError("Please upload cars_ds_final.csv")
    DATA_FILE = csv_files[0]


df = pd.read_csv(DATA_FILE)


print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
print("\nFirst five rows:")
display(df.head())
print("\nMissing values:")
display(df.isnull().sum().to_frame("Missing count"))
print("\nDuplicate rows:", df.duplicated().sum())
display(df.describe(include="all").T)



print("Missing values before cleaning:")
display(df.isnull().sum().to_frame("Missing count"))
print("Duplicate rows before cleaning:", df.duplicated().sum())


df = df.drop_duplicates().copy()


if "Unnamed: 0" in df.columns:
    df = df.drop(columns=["Unnamed: 0"])


target = "Ex-Showroom_Price"


if target not in df.columns:
    raise ValueError("The dataset must contain an 'Ex-Showroom_Price' column.")


# Convert prices such as 'Rs. 2,92,667' into numeric INR values.
df[target] = (
    df[target].astype(str)
    .str.replace(",", "", regex=False)
    .str.replace("Rs.", "", regex=False)
    .str.extract(r"(\d+(?:\.\d+)?)")[0]
)
df[target] = pd.to_numeric(df[target], errors="coerce")
```

```
df = df.dropna(subset=[target])
df = df[df[target] > 0].copy()


# Use meaningful car specifications available in this dataset.
numeric_candidates = [
    "Displacement", "Cylinders", "Valves_Per_Cylinder",
    "Fuel_Tank_Capacity", "Height", "Length", "Width", "Doors",
    "City_Mileage", "Highway_Mileage", "ARAI_Certified_Mileage",
    "Kerb_Weight", "Gears", "Ground_Clearance", "Power", "Torque",
    "Seating_Capacity", "Wheelbase", "Boot_Space",
    "Number_of_Airbags", "Electric_Range"
]


categorical_candidates = [
    "Make", "Model", "Variant", "Fuel_Type", "Body_Type",
    "Type", "Drivetrain", "Cylinder_Configuration", "Emission_Norm"
]


numeric_features = [
    col for col in numeric_candidates
    if col in df.columns and df[col].notna().any()
]
categorical_features = [
    col for col in categorical_candidates
    if col in df.columns and df[col].notna().any()
]


# Extract the first numeric value from specifications that include units.
# Examples: '624 cc' -> 624, '23.6 km/litre' -> 23.6,
# '38PS@5500rpm' -> 38, and '51Nm@4000rpm' -> 51.
for col in numeric_features:
    df[col] = pd.to_numeric(
        df[col].astype(str).str.extract(r"([-+]?\d*\.?\d+)")[0],
        errors="coerce"
    )


features = numeric_features + categorical_features
if not features:
    raise ValueError("No usable car specification columns were found.")


X = df[features].copy()
y = df[target].copy()


print("Rows after cleaning:", len(df))
print("Selected features:", features)
print("Numeric features:", numeric_features)
print("Categorical features:", categorical_features)
print("\nMissing feature values:")
display(X.isnull().sum().to_frame("Missing count"))



print("Descriptive statistics for numerical columns:")
display(df[numeric_features + [target]].describe().T)


print("\nCar selling price summary:")
print("Mean selling price (INR):", round(df[target].mean(), 2))
print("Median selling price (INR):", round(df[target].median(), 2))
print("Mode selling price (INR):", round(df[target].mode().iloc[0], 2))
print("Variance:", round(df[target].var(), 2))
print("Standard deviation:", round(df[target].std(), 2))
print("Minimum selling price (INR):", round(df[target].min(), 2))
print("Maximum selling price (INR):", round(df[target].max(), 2))


# Histogram of new-car ex-showroom prices
plt.figure(figsize=(8, 4))
plt.hist(df[target], bins=30, edgecolor="black")
plt.title("New-Car Ex-Showroom Price Distribution")
plt.xlabel("Ex-Showroom Price (INR)")
plt.ylabel("Count")
plt.show()


# Boxplot of new-car ex-showroom prices
plt.figure(figsize=(8, 4))
plt.boxplot(df[target].dropna(), vert=False)
plt.title("New-Car Ex-Showroom Price Boxplot")
plt.xlabel("Ex-Showroom Price (INR)")
plt.show()


# Scatterplots for available numerical car specifications
for feature in ["Displacement", "Seating_Capacity", "ARAI_Certified_Mileage",
                "Kerb_Weight", "Power", "Torque"]:
    if feature in numeric_features and df[feature].notna().sum() >= 3:
        plt.figure(figsize=(7, 4))
        plt.scatter(df[feature], df[target], alpha=0.5)
        plt.title(f"Ex-Showroom Price vs {feature}")
```

```
        plt.xlabel(feature)
        plt.ylabel("Ex-Showroom Price (INR)")
        plt.show()

# Compare median prices by available categories
for feature in ["Make", "Fuel_Type", "Type", "Body_Type"]:
    if feature in df.columns:
        category_prices = (
            df.groupby(feature)[target].median()
            .sort_values(ascending=False).head(20)
        )
        plt.figure(figsize=(9, 4))
        plt.bar(category_prices.index.astype(str), category_prices.values)
        plt.title(f"Median Ex-Showroom Price by {feature}")
        plt.xlabel(feature)
        plt.ylabel("Median Ex-Showroom Price (INR)")
        plt.xticks(rotation=35, ha="right")
        plt.tight_layout()
        plt.show()

# Correlation heatmap for numerical features
correlation_columns = numeric_features + [target]
correlation = df[correlation_columns].corr()
plt.figure(figsize=(10, 7))
plt.imshow(correlation, aspect="auto", vmin=-1, vmax=1)
plt.colorbar()
plt.xticks(range(len(correlation.columns)), correlation.columns, rotation=45, ha="right")
plt.yticks(range(len(correlation.index)), correlation.index)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()



# Observed car selling-price distribution
plt.figure(figsize=(7, 4))
plt.hist(df[target], bins=30, edgecolor="black")
plt.title("Observed Car Selling Price Distribution")
plt.xlabel("Selling Price (INR)")
plt.show()

# Simulated uniform distribution (demonstration only)
uniform_data = np.random.uniform(low=0, high=1, size=1000)
plt.figure(figsize=(7, 4))
plt.hist(uniform_data, bins=20, edgecolor="black")
plt.title("Simulated Uniform Distribution")
plt.show()

# Simulated binomial distribution (demonstration only)
binomial_data = np.random.binomial(n=10, p=0.5, size=1000)
plt.figure(figsize=(7, 4))
plt.hist(binomial_data, bins=11, edgecolor="black")
plt.title("Simulated Binomial Distribution")
plt.show()

# CLT: repeated samples from car selling prices
sample_means = []
for i in range(1000):
    sample = df[target].sample(n=min(30, len(df)), replace=True)
    sample_means.append(sample.mean())

plt.figure(figsize=(7, 4))
plt.hist(sample_means, bins=30, edgecolor="black")
plt.title("Sampling Distribution of Mean Car Selling Price")
plt.xlabel("Sample Mean (INR)")
plt.show()

# Simple random sample
simple_sample = df.sample(n=min(100, len(df)), random_state=42)
print("Simple random sample size:", len(simple_sample))
display(simple_sample.head())

# Stratified sample by fuel type
if "Fuel_Type" in df.columns:
    stratified_sample = (
        df.groupby("Fuel_Type", group_keys=False)
          .apply(lambda group: group.sample(n=min(20, len(group)), random_state=42))
          .reset_index(drop=True)
    )
    print("Fuel-stratified sample size:", len(stratified_sample))
    display(stratified_sample.head())
else:
    print("Fuel_Type column is not available; stratified sampling was skipped.")

standard_error = stats.sem(df[target])
print("Standard error of mean new-car ex-showroom price:", round(standard_error, 2), "INR")
```

```
confidence = 0.95
mean_price = df[target].mean()
standard_error = stats.sem(df[target])


ci_low, ci_high = stats.t.interval(
    confidence,
    df=len(df) - 1,
    loc=mean_price,
    scale=standard_error
)


print(f"Mean car selling price: Rs.{mean_price:,.2f}")
print(f"95% confidence interval: Rs.{ci_low:,.2f} to Rs.{ci_high:,.2f}")


alpha = 0.05


# A. One-sample t-test: compare mean ex-showroom price with a reference price
reference_price = 1000000  # Editable reference price in INR
t_stat, p_value = stats.ttest_1samp(df[target], popmean=reference_price)


print("ONE-SAMPLE T-TEST")
print(f"H0: Mean new-car ex-showroom price = Rs.{reference_price:,.0f}")
print(f"t-statistic = {t_stat:.3f}, p-value = {p_value:.5f}")
print("Decision:", "Reject H0" if p_value < alpha else "Fail to reject H0")


# B. Independent t-test: compare manual and automatic transmission prices
if "Type" in df.columns:
    manual_prices = df.loc[df["Type"].astype(str).str.contains("manual", case=False, na=False), target]
    automatic_prices = df.loc[df["Type"].astype(str).str.contains("automatic|amt|cvt|dct", case=False, na=False),
        target]


    if len(manual_prices) >= 2 and len(automatic_prices) >= 2:
        t_stat, p_value = stats.ttest_ind(
            manual_prices, automatic_prices, equal_var=False
        )
        print("\nINDEPENDENT TWO-SAMPLE T-TEST")
        print("H0: Mean prices are equal for manual and automatic cars.")
        print(f"t-statistic = {t_stat:.3f}, p-value = {p_value:.5f}")
        print("Decision:", "Reject H0" if p_value < alpha else "Fail to reject H0")
    else:
        print("\nNot enough manual/automatic records for the independent t-test.")


# C. One-way ANOVA: compare prices across fuel types
if "Fuel_Type" in df.columns:
    fuel_groups = [
        group[target].dropna()
        for _, group in df.groupby("Fuel_Type")
        if group[target].notna().sum() >= 2
    ]
    if len(fuel_groups) >= 2:
        f_stat, p_value = stats.f_oneway(*fuel_groups)
        print("\nONE-WAY ANOVA")
        print("H0: Mean new-car prices are equal across fuel types.")
        print(f"F-statistic = {f_stat:.3f}, p-value = {p_value:.5f}")
        print("Decision:", "Reject H0" if p_value < alpha else "Fail to reject H0")


# D. Chi-square test: fuel type and transmission type
if "Fuel_Type" in df.columns and "Type" in df.columns:
    contingency = pd.crosstab(df["Fuel_Type"], df["Type"])
    if contingency.shape[0] >= 2 and contingency.shape[1] >= 2:
        chi2, p_value, dof, expected = stats.chi2_contingency(contingency)
        print("\nCHI-SQUARE TEST OF INDEPENDENCE")
        print("H0: Fuel type and transmission type are independent.")
        print(f"Chi-square = {chi2:.3f}, p-value = {p_value:.5f}")
        print("Decision:", "Reject H0" if p_value < alpha else "Fail to reject H0")



numeric_predictors = [
    col for col in numeric_features
    if df[col].notna().sum() >= 3 and df[col].nunique(dropna=True) > 1
]


print("Pearson correlation matrix:")
display(df[numeric_predictors + [target]].corr(method="pearson"))


print("\nCovariance matrix:")
display(df[numeric_predictors + [target]].cov())


for x_col in numeric_predictors:
    pair = df[[x_col, target]].dropna()
    if len(pair) >= 3 and pair[x_col].nunique() > 1 and pair[target].nunique() > 1:
        pearson_r, pearson_p = stats.pearsonr(pair[x_col], pair[target])
        spearman_r, spearman_p = stats.spearmanr(pair[x_col], pair[target])
        print(f"\n{x_col} vs {target}")
        print(f"Pearson r = {pearson_r:.3f}, p-value = {pearson_p:.5f}")
        print(f"Spearman rho = {spearman_r:.3f}, p-value = {spearman_p:.5f}")
```

```
X = df[features].copy()
y = df[target].copy()


# Keep the same feature types selected during data cleaning.
numeric_features = X.select_dtypes(include="number").columns.tolist()
categorical_features = X.select_dtypes(include=["object", "category", "bool"]).columns.tolist()


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)


print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)


numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])


categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])


transformers = []
if numeric_features:
    transformers.append(("num", numeric_transformer, numeric_features))
if categorical_features:
    transformers.append(("cat", categorical_transformer, categorical_features))


preprocessor = ColumnTransformer(transformers=transformers)



models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Lasso Regression": Lasso(alpha=0.1, max_iter=10000),
    "Decision Tree": DecisionTreeRegressor(max_depth=8, random_state=42),
    "Random Forest": RandomForestRegressor(
        n_estimators=200, max_depth=12, random_state=42, n_jobs=-1
    )
}


results = []
trained_models = {}
predictions_by_model = {}


for name, regressor in models.items():
    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("model", regressor)
    ])


    pipeline.fit(X_train, y_train)
    predictions = pipeline.predict(X_test)


    trained_models[name] = pipeline
    predictions_by_model[name] = predictions


    mae = mean_absolute_error(y_test, predictions)
    mse = mean_squared_error(y_test, predictions)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, predictions)


    results.append([name, mae, mse, rmse, r2])


results_df = pd.DataFrame(
    results,
    columns=["Model", "MAE", "MSE", "RMSE", "R2 Score"]
)


print("MODEL COMPARISON")
display(results_df.round(2).sort_values("R2 Score", ascending=False))


best_index = results_df["R2 Score"].idxmax()
best_model_name = results_df.loc[best_index, "Model"]
best_model = trained_models[best_model_name]
best_predictions = predictions_by_model[best_model_name]


t_stat, p_value = stats.ttest_rel(
    y_test.to_numpy(),
```

```
    best_predictions
)


print("PAIRED T-TEST: Actual vs Predicted Car Price")
print(f"Mean actual price: Rs.{y_test.mean():,.2f}")
print(f"Mean predicted price: Rs.{np.mean(best_predictions):,.2f}")
print(f"t-statistic = {t_stat:.3f}, p-value = {p_value:.5f}")
print(
    "Decision:",
    "Reject H0: mean paired difference is not zero"
    if p_value < alpha
    else "Fail to reject H0: insufficient evidence of a mean difference"
)


best_index = results_df["R2 Score"].idxmax()
best_model_name = results_df.loc[best_index, "Model"]
best_model = trained_models[best_model_name]


print("Best Model:", best_model_name)
print("Best R2 Score:", round(results_df.loc[best_index, "R2 Score"], 4))


# Save the complete fitted pipeline (preprocessing + regression model).
# The Streamlit app will load this exact file.
joblib.dump(best_model, "new_car_price_model.pkl")


# Save the selected model name.
with open("best_car_model_name.txt", "w", encoding="utf-8") as file:
    file.write(best_model_name)


# Save the exact feature order used during training.
with open("car_model_features.json", "w", encoding="utf-8") as file:
    json.dump(features, file, indent=4)


# Save a Streamlit-friendly input configuration.
feature_config = {}


for col in numeric_features:
    values = X[col].dropna()
    feature_config[col] = {
        "type": "number",
        "default": float(values.median()),
        "min": float(values.min()),
        "max": float(values.max())
    }


for col in categorical_features:
    options = sorted(X[col].dropna().astype(str).unique().tolist())
    feature_config[col] = {
        "type": "category",
        "options": options,
        "default": options[0] if options else "Unknown"
    }


with open("car_feature_config.json", "w", encoding="utf-8") as file:
    json.dump(feature_config, file, indent=4)


# Save the scikit-learn version to prevent the .pkl version mismatch.
with open("sklearn_version.txt", "w", encoding="utf-8") as file:
    file.write(sklearn.__version__)


# Save only selected model features and the target.
df[features + [target]].to_csv("cars_cleaned.csv", index=False)
results_df.to_csv("car_model_comparison.csv", index=False)


print("\nFiles created successfully:")
print("new_car_price_model.pkl")
print("best_car_model_name.txt")
print("car_model_features.json")
print("car_feature_config.json")
print("sklearn_version.txt")
print("cars_cleaned.csv")
print("car_model_comparison.csv")



plt.figure(figsize=(7, 5))
plt.scatter(y_test, best_predictions, alpha=0.5)
minimum = min(y_test.min(), np.min(best_predictions))
maximum = max(y_test.max(), np.max(best_predictions))
plt.plot([minimum, maximum], [minimum, maximum], "r--")
plt.xlabel("Actual Ex-Showroom Price (INR)")
plt.ylabel("Predicted Ex-Showroom Price (INR)")
plt.title(f"Actual vs Predicted New-Car Prices - {best_model_name}")
plt.show()


plt.figure(figsize=(9, 4))
```

```
plt.bar(results_df["Model"], results_df["R2 Score"])
plt.title("R2 Score Comparison")
plt.xticks(rotation=25)
plt.tight_layout()
plt.show()


print("Best model performance:")
display(results_df.loc[[best_index]].round(2))



from google.colab import files
import os


files_to_download = [
    "new_car_price_model.pkl",
    "best_car_model_name.txt",
    "car_model_features.json",
    "car_feature_config.json",
    "sklearn_version.txt",
    "cars_cleaned.csv",
    "car_model_comparison.csv"
]


for file_name in files_to_download:
    if os.path.exists(file_name):
        files.download(file_name)
    else:
        print("File not found:", file_name)
```

# 6. RESULTS AND DISCUSSION

## 6.1 Output Screenshots and Results

**Data loading and preprocessing output:** the dataset has 1,276 rows and 141 columns with 0 duplicate rows (Figure 8). After cleaning, all 1,276 rows are retained with 21 numeric and 9 categorical features (Figure 9). After the train-test split the training set has 1,020 rows and the test set has 256 rows (Figure 13).

```
df = pd.read_csv(DATA_FILE)

print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
print("\nDuplicate rows:", df.duplicated().sum())
```

```
Dataset shape: (1276, 141)

Column names:
['Unnamed: 0', 'Make', 'Model', 'Variant', 'Ex-Showroom_Price', 'Displacement', 'Cylinders',
    'Valves_Per_Cylinder', 'Drivetrain', 'Cylinder_Configuration', 'Emission_Norm', 'Engine_Location',
    'Fuel_System', 'Fuel_Tank_Capacity', 'Fuel_Type', 'Height', 'Length', 'Width', 'Body_Type', ... ]

Duplicate rows: 0
```

*Figure 8: Data loading output (shape, columns and duplicates)*

```
print("Duplicate rows before cleaning:", df.duplicated().sum())
...
print("Rows after cleaning:", len(df))
print("Numeric features:", numeric_features)
print("Categorical features:", categorical_features)
```

```
Duplicate rows before cleaning: 0
Rows after cleaning: 1276
Numeric features: ['Displacement', 'Cylinders', 'Valves_Per_Cylinder', 'Fuel_Tank_Capacity', 'Height',
    'Length', 'Width', 'Doors', 'City_Mileage', 'Highway_Mileage', 'ARAI_Certified_Mileage',
    'Kerb_Weight', 'Gears', 'Ground_Clearance', 'Power', 'Torque', 'Seating_Capacity', 'Wheelbase',
    'Boot_Space', 'Number_of_Airbags', 'Electric_Range']
Categorical features: ['Make', 'Model', 'Variant', 'Fuel_Type', 'Body_Type', 'Type', 'Drivetrain',
    'Cylinder_Configuration', 'Emission_Norm']
```

*Figure 9: Data cleaning output (rows retained and selected features)*

**Statistical inference output:** Figure 10 shows the descriptive statistics of the price (mean 4,596,537.89, median 1,060,064.50, mode 999,900, standard deviation 12,147,346.94 INR); Figure 11 shows the standard error and the 95 % confidence interval; Figure 12 shows the four hypothesis-test results.

```
print("Car selling price summary:")
print("Mean selling price (INR):", round(df[target].mean(), 2))
print("Median selling price (INR):", round(df[target].median(), 2))
print("Mode selling price (INR):", round(df[target].mode().iloc[0], 2))
print("Variance:", round(df[target].var(), 2))
print("Standard deviation:", round(df[target].std(), 2))
print("Minimum selling price (INR):", round(df[target].min(), 2))
print("Maximum selling price (INR):", round(df[target].max(), 2))
```

```
Car selling price summary:
Mean selling price (INR): 4596537.89
Median selling price (INR): 1060064.5
Mode selling price (INR): 999900
Variance: 147558037798510.47
Standard deviation: 12147346.94
Minimum selling price (INR): 236447
Maximum selling price (INR): 212155397
```

*Figure 10: Descriptive statistics of the ex-showroom price*

```
standard_error = stats.sem(df[target])
print("Standard error of mean new-car ex-showroom price:", round(standard_error, 2), "INR")
```

```
... Standard error of mean new-car ex-showroom price: 340060.43 INR
```

```
confidence = 0.95
ci_low, ci_high = stats.t.interval(confidence, df=len(df) - 1, loc=mean_price, scale=standard_error)

print(f"Mean car selling price: ₹{mean_price:,.2f}")
print(f"95% confidence interval: ₹{ci_low:,.2f} to ₹{ci_high:,.2f}")
```

```
... Mean car selling price: ₹4,596,537.89
    95% confidence interval: ₹3,929,398.38 to ₹5,263,677.39
```

*Figure 11: Standard error and 95 % confidence interval of the mean price*

```
alpha = 0.05
reference_price = 1000000  # Editable reference price in INR
t_stat, p_value = stats.ttest_1samp(df[target], popmean=reference_price)
...  # independent t-test, one-way ANOVA and chi-square test follow
```

```
... ONE-SAMPLE T-TEST
    H0: Mean new-car ex-showroom price = ₹1,000,000
    t-statistic = 10.576, p-value = 0.00000
    Decision: Reject H0

    INDEPENDENT TWO-SAMPLE T-TEST
    H0: Mean prices are equal for manual and automatic cars.
    t-statistic = -11.425, p-value = 0.00000
    Decision: Reject H0

    ONE-WAY ANOVA
    H0: Mean new-car prices are equal across fuel types.
    F-statistic = 7.179, p-value = 0.00000
    Decision: Reject H0

    CHI-SQUARE TEST OF INDEPENDENCE
    H0: Fuel type and transmission type are independent.
    Chi-square = 76.832, p-value = 0.00000
    Decision: Reject H0
```

*Figure 12: Hypothesis test output (one-sample t, two-sample t, ANOVA, chi-square)*

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
... Training data: (1020, 30)
    Testing data: (256, 30)
```

*Figure 13: Train-test split output*

**Model comparison and performance results:** Table 6 and Figure 6 present the metrics of all five models, and Figure 14 shows the notebook output. The Decision Tree is first, followed closely by the Random Forest, with the linear models behind.

```
results_df = pd.DataFrame(results, columns=["Model", "MAE", "MSE", "RMSE", "R2 Score"])

print("MODEL COMPARISON")
display(results_df.round(2).sort_values("R2 Score", ascending=False))
```

```
MODEL COMPARISON
                Model         MAE           MSE        RMSE  R2 Score
3       Decision Tree   935334.29  9.929937e+12  3151180.20      0.95
4       Random Forest   783042.22  1.122660e+13  3350612.14      0.94
1    Ridge Regression  1707850.34  1.292605e+13  3595280.95      0.93
0   Linear Regression  1830025.15  2.292713e+13  4788228.16      0.88
2    Lasso Regression  1991447.13  3.169035e+13  5629418.13      0.84
```

*Figure 14: Model comparison output*

**Best model selection:** the Decision Tree was selected automatically (highest $R^2 = 0.9485$), the paired t-test gave $p = 0.24787$, and the model files were created (Figure 15). Figure 16 compares actual and predicted prices for the test set; most points lie close to the red diagonal line, which indicates accurate predictions, while the highest-priced car is the most difficult to predict.

```
t_stat, p_value = stats.ttest_rel(y_test.to_numpy(), best_predictions)

print("PAIRED T-TEST: Actual vs Predicted Car Price")
print(f"Mean actual price: ₹{y_test.mean():,.2f}")
print(f"Mean predicted price: ₹{np.mean(best_predictions):,.2f}")
print(f"t-statistic = {t_stat:.3f}, p-value = {p_value:.5f}")
```

```
PAIRED T-TEST: Actual vs Predicted Car Price
Mean actual price: ₹4,166,582.38
Mean predicted price: ₹4,394,532.69
t-statistic = -1.158, p-value = 0.24787
Decision: Fail to reject H0: insufficient evidence of a mean difference
```

```
best_index = results_df["R2 Score"].idxmax()
best_model_name = results_df.loc[best_index, "Model"]
best_model = trained_models[best_model_name]

print("Best Model:", best_model_name)
print("Best R2 Score:", round(results_df.loc[best_index, "R2 Score"], 4))
joblib.dump(best_model, "new_car_price_model.pkl")
```

```
Best Model: Decision Tree
Best R2 Score: 0.9485

Files created successfully:
new_car_price_model.pkl
best_car_model_name.txt
car_model_features.json
car_feature_config.json
sklearn_version.txt
cars_cleaned.csv
car_model_comparison.csv
```

*Figure 15: Paired t-test, best model selection and saved files*

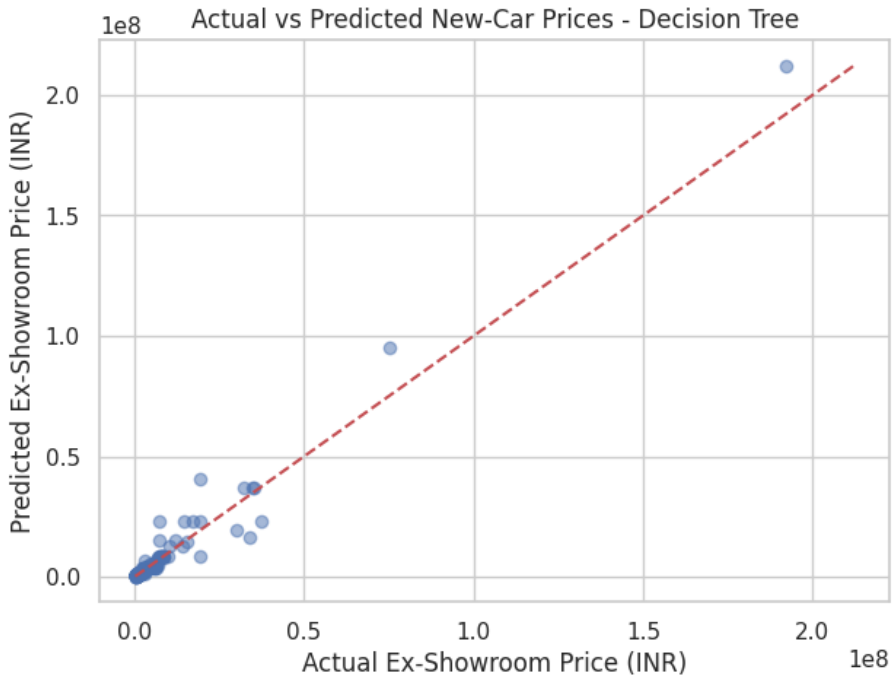



*Figure 16: Actual vs predicted ex-showroom price for the Decision Tree model (test set)*

GUI interface and prediction results: see Figure 7 in Section 4.3.

## 6.2 Result Analysis

- **Model differences:** the linear models ($R^2$ between 0.84 and 0.93) cannot fully capture the non-linear influence of brand, engine size and body type on price. The tree-based models improve this considerably: the Decision Tree is best ($R^2 = 0.9485$, RMSE = 3,151,180 INR) and the Random Forest is nearly as good ($R^2 = 0.94$) with the lowest MAE (783,042 INR).
- **Strengths:** high explanatory power on unseen test data, a single pipeline that includes preprocessing, statistical evidence that specifications such as power, torque, displacement, cylinders and fuel type are significantly related to price, automatic model selection, and exported files that make the GUI simple to build.
- **Errors:** the RMSE (about 3.15 million INR) is more than three times the MAE (about 0.94 million INR), which shows that a small number of very expensive cars produce large errors. The MAE is also large relative to a typical car (the median price is about 10.6 lakh INR), so the model is much less precise for ordinary budget cars than the $R^2$ suggests.
- **Statistical findings:** all four hypothesis tests rejected their null hypotheses ($\alpha = 0.05$): the mean price differs from Rs. 10 lakh, differs between manual and automatic cars and across fuel types, and fuel type is associated with transmission type. Statistical significance does not automatically mean practical importance, and the large sample variation caused by luxury cars should be kept in mind.
- **Possible issues in the data and evaluation:** the price distribution is extremely skewed, and a handful of supercars (for example the two cars priced above Rs. 19 crore) dominate the squared-error metrics and the correlation values. Some columns contain inconsistent units, Electric_Range is available for only 17 variants but is median-imputed for all rows, and the high-cardinality Model and Variant features (1,064 distinct variants among 1,276 rows) let a tree memorise

individual cars. The test set has only 256 rows, so the metrics can change noticeably with a different split.

- **Limitations:** the model estimates historical ex-showroom prices in this dataset and not current live prices; it uses a single random train-test split and fixed hyperparameters without tuning or cross-validation; and it cannot be expected to predict well for new brands or specifications that are not represented in the data.
- **Objectives:** all objectives were achieved: the data was explored and cleaned, statistical inference was applied, five regression models were trained and compared, the best model was selected and saved, and the files required by the Streamlit GUI were exported.

## 7. CONCLUSION

This project developed a New Car Price Prediction System using a dataset of 1,276 new-car variants with 30 selected specifications. The statistical analysis showed a mean ex-showroom price of Rs. 4,596,537.89 (95 % confidence interval Rs. 3,929,398.38 to Rs. 5,263,677.39), a strongly right-skewed price distribution, and significant relationships between price and specifications such as power, torque, displacement, fuel type and transmission type. After preprocessing with imputation, scaling and one-hot encoding, five regression models (Linear, Ridge, Lasso, Decision Tree and Random Forest) were trained and compared using MAE, MSE, RMSE and $R^2$. The Decision Tree achieved the best $R^2$ (0.9485, MAE = 935,334.29 INR, RMSE = 3,151,180.20 INR) and was selected as the final model and saved as new_car_price_model.pkl, with the Random Forest a very close second.

The main limitations are the sparse and sometimes inconsistent specification data, the extreme skew caused by luxury cars, the use of a single random train-test split, and the fact that predictions refer to historical ex-showroom prices. Possible improvements include correcting unit inconsistencies, modelling the logarithm of the price or treating luxury cars separately, hyperparameter tuning and cross-validation, trying gradient-boosting models (XGBoost, LightGBM), adding more recent price data, and deploying the Streamlit application online.

## 8. GITHUB LINK

GitHub Repository URL:

https://github.com/24thakreo/New-Car-Price-Prediction.git

The repository should contain app.py, the Colab notebook (.ipynb) and its PDF export, the trained model file new_car_price_model.pkl, and the supporting files car_feature_config.json, car_model_features.json, sklearn_version.txt, best_car_model_name.txt, cars_cleaned.csv and car_model_comparison.csv.

## 9. REFERENCES

[1] Indian Cars dataset (cars_ds_final.csv): new-car models and variants with ex-showroom prices and specifications, as supplied with the project notebook.

[2] Scikit-learn documentation. https://scikit-learn.org/stable/

[3] SciPy statistics (scipy.stats) documentation. https://docs.scipy.org/doc/scipy/reference/stats.html

[4] Streamlit documentation. https://docs.streamlit.io/

[5] Pandas documentation. https://pandas.pydata.org/docs/

[6] Joblib documentation. https://joblib.readthedocs.io/

[7] Matplotlib and Seaborn documentation. https://matplotlib.org/ and https://seaborn.pydata.org/